Full-Stack Engineering Training — Day 8 Study Material

MODULE	TOPIC	STACK
Frontend Engineering (Next.js)	App Router, Server Components & Dynamic Routes	Next.js 15+ • React 19 • TypeScript

> **Key Learning Objectives:** Understand why Next.js is preferred over traditional React SPAs for high-performance enterprise applications, master directory-based routing conventions (`page.tsx` , `layout.tsx` , `loading.tsx` , `error.tsx`), differentiate between React Server Components (RSC) and Client Components (`'use client'`), and build dynamic routes with query parameters.

1. Mental Model: Why Next.js Over Plain React?

Traditional Single Page Applications (built with Vite or Create React App) send a blank HTML shell to the browser. The client must download several megabytes of JavaScript before anything renders. On construction job sites where network connectivity is unpredictable, this results in slow white screens and high battery drain.

> **Next.js Server-First Advantage:**
> - Pre-renders full HTML on the server for immediate display (sub-second First Contentful Paint).
> - Zero JavaScript bundle overhead for static parts of the page.
> - Search engines and social crawlers can index complete page content without executing JavaScript.

2. App Router Directory Structure & Special Files

Folders define URL paths, while standardized filenames define behavior and boundaries:

Filename	Role & Purpose	Component Type
`page.tsx`	Unique view rendered at that URL route segment	Server Component by default
`layout.tsx`	Shared shell (sidebar, navigation, header) persisting state on nav	Server Component by default
`loading.tsx`	Instant loading skeleton using React Suspense stream	Server Component
`error.tsx`	Client error boundary catching unhandled child runtime errors	**Must be 'use client'**
`not-found.tsx`	404 page rendered when `notFound()` is called	Server Component

3. Server Components (RSC) vs Client Components ('use client')

In the App Router, every component is a Server Component unless marked with `'use client'`.

Feature	Server Component (RSC)	Client Component ('use client')
Default?	Yes (automatic in app/ directory)	No (requires 'use client' at line 1)
Where It Runs	Server only (never ships to browser)	Server (initial render) + Browser (hydration)
JS Bundle Size	**0 KB** (Zero client bundle weight)	Adds to client JavaScript bundle
React Hooks	No (`useState` , `useEffect` forbidden)	Yes (all React hooks allowed)
Browser APIs	No (`window` , `localStorage` undefined)	Yes (full DOM access in `useEffect`)
Data Fetching	Direct async/await to database or API	fetch() inside useEffect or TanStack Query

Server Component Example:

```
// app/projects/page.tsx (Server Component)
export default async function ProjectsPage() {
  // Direct async data fetching on the server — zero client JS!
  const res = await fetch('https://api.example.com/projects', { cache: 'no-store' });
  const projects = await res.json();

  return (
    <div className="project-grid">
      {projects.map(p => (
        <div key={p.id} className="card">
          <h3>{p.name}</h3>
          <p>Status: {p.status}</p>
        </div>
      ))}
    </div>
  );
}
```

Client Component Example:

```
// components/FilterDropdown.tsx
'use client'; // Required for interactive state and event handlers

import { useState } from 'react';

export default function FilterDropdown() {
  const [filter, setFilter] = useState('ALL');

  return (
    <select value={filter} onChange={(e) => setFilter(e.target.value)}>
      <option value="ALL">All Projects</option>
      <option value="ACTIVE">Active</option>
      <option value="COMPLETED">Completed</option>
    </select>
  );
}
```

4. Dynamic Routes and Search Parameters

To handle variable parameters like IDs, use folder names enclosed in square brackets: `[id]` . In modern Next.js, `params` and `searchParams` are asynchronous Promises:

```
// app/projects/[id]/page.tsx
interface PageProps {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ tab?: string }>;
}

export default async function ProjectDetailPage({ params, searchParams }: PageProps) {
  const { id } = await params;
  const { tab } = await searchParams;

  return (
    <div>
      <h1>Project Details: {id}</h1>
      <p>Active Tab: {tab || 'overview'}</p>
    </div>
  );
}
```

5. Common Beginner Pitfalls & "Intern Watch-Outs"

> **1. Marking Every File with 'use client':**
> Putting `'use client'` at the top of every file negates all performance benefits of Next.js, turning it back into a slow SPA. *Best Practice:* Keep pages and layouts as Server Components, and extract only small interactive elements (buttons, inputs) into Client Components.

> **2. Forgetting to await params:**
> In Next.js 15+, `params` is a Promise. Destructuring it directly without `await` causes console warnings and missing parameters.

> **3. Using Standard <a> tags:**
> Standard anchor tags cause full-page browser reloads, discarding React state. Always use `<Link href="...">` from `next/link`.

6. Quick Memory Cheat Sheet

- **P-L-L-E Architecture**: **P**age (UI), **L**ayout (Wrapper), **L**oading (Suspense), **E**rror (Boundary).
- **Server-First Rule**: Fetch data on the server; pass data down to client islands as props.
- **Client Islands Rule**: Add `'use client'` only when you need `useState`, `useEffect`, or click events.

7. Verification & Exercise Output (o/p)

```
--- RUNNING DAY 8 NEXT.JS APP ROUTER VERIFICATION ---
✓ Root layout rendered successfully with navigation sidebar
✓ Dashboard page rendered at / with key metric cards
✓ Dynamic route /projects/[id] extracted param { id: 'p101' }
✓ Nested route /projects/[id]/towers/[towerId] resolved properly
✓ Search parameters parsed ?status=ACTIVE correctly
ALL 5 VERIFICATIONS PASSED CLEANLY!
```